

The Vanishing Middle: Developmental Co-Building in the Age of Autonomous AI Agents

Anthony Paterson

Fractal Media Infrastructure

Public preprint | 5 October 2026

Abstract

Rapid advances in agentic artificial intelligence increasingly make it possible to delegate substantial software-development tasks from an initial specification through implementation, testing, and revision. This capability offers significant benefits, particularly where the objective is efficient artifact production. However, the transition from manual software development to autonomous agentic construction risks obscuring an intermediate mode of practice: sustained human-AI co-building.

In this mode, the human need not possess conventional implementation fluency, while also remaining substantially more involved than a requester supervising autonomous output. Through repeated participation in decomposition, architectural discussion, implementation, debugging, inspection, correction, and recombination, users may acquire forms of systems literacy that allow them to reason about components, boundaries, interactions, limitations, and transferable patterns without becoming conventional programmers. Such knowledge can subsequently alter the character of human contribution: the user becomes increasingly capable of identifying structural analogies, recombining established mechanisms in novel contexts, specifying missing components, and directing AI implementation toward architectures that neither an initial high-level request nor familiar implementation patterns necessarily imply.

This paper argues that increasingly autonomous AI development may create a **vanishing middle**. As human participation becomes less instrumentally necessary for producing functional artifacts, the developmental value of participation may become less visible. New users may consequently encounter AI-assisted development as a binary choice between conventional programming and autonomous delegation, without recognising sustained co-building as a distinct option with different benefits.

The argument is not that co-building should be preferred universally, nor that autonomous delegation is undesirable. Rather, the paper argues for preserving the **legibility of the middle**: maintaining conceptual, cultural, and interface support for users who deliberately choose participation because the construction process itself produces valuable human capability. Rapid technological advancement should not cause this option to disappear before its distinctive properties are adequately understood.

1. The Disappearing Transition

Historically, producing software required substantial implementation capability. Generative coding systems weakened that requirement by allowing people with limited programming ability to participate in increasingly sophisticated construction through natural-language interaction.

That created an unusual intermediate condition.

AI possessed enough technical competence to extend human capability substantially, while retaining enough limitations that iterative human participation remained valuable. A user could therefore begin building beyond their existing implementation competence while remaining exposed to the architecture, decomposition, debugging, constraints, and trade-offs involved in producing the artifact.

Agentic systems increasingly weaken the second condition.

The same user may now be able to specify an intended artifact and delegate much of the intermediate construction process. From the perspective of artifact production, this represents substantial progress.

From the perspective of **capability acquisition through participation**, however, something else may be occurring.

The transitional practice through which a non-programmer could build *and simultaneously learn how increasingly complex systems fit together* may cease to be instrumentally necessary before its developmental properties have become widely recognised.

The transition can therefore be represented crudely as:

Implementation prerequisite

learn substantial technical skill -> build

v

Developmental co-building

build with AI -> acquire systems literacy through participation -> build increasingly sophisticated systems with AI

v

Autonomous construction

specify desired artifact -> delegate construction -> evaluate result

The third mode does not invalidate the second. It changes the conditions that previously caused users to encounter it.

1.1 Relation to Established Literature

The proposed middle is not offered as if no neighbouring concepts exist. Several established traditions already describe important pieces of the phenomenon. **Situated cognition** and **cognitive apprenticeship** emphasise that knowledge develops through participation in authentic activity, with modelling, scaffolding, coaching, articulation, reflection, and the gradual transfer of responsibility playing central roles in learning (Brown, Collins, & Duguid, 1989; Collins, Brown, & Holum, 1991). Papert's constructionist tradition similarly treats making and iterative engagement with computational artefacts as a route to learning rather than merely as a means of producing outputs (Papert, 1980).

In human-computer interaction, **end-user development** and **meta-design** have long examined how people without conventional programming expertise can modify systems, become co-developers, and participate in the ongoing design of computational environments (Lieberman, Paternò, & Wulf, 2006; Fischer & Giaccardi, 2006). **Mixed-initiative interaction** likewise rejects a simple choice between direct human manipulation and autonomous agents, instead examining how initiative and control can move between human and machine (Horvitz, 1999).

The present paper draws these traditions together around a more specific question created by rapidly improving code-generating agents: **what happens when participation ceases to be instrumentally necessary before its developmental effects are well understood?** Recent research on AI-assisted programming already shows a mixed picture. Reviews identify both educational opportunities and risks of over-reliance (Cambaz & Zhang, 2024). Controlled studies show that AI code generation can improve short-term efficiency and reduce workload for novices (Gardella, Pettit, & Riggs, 2024), while other work shows that beginning programmers can still struggle to specify intent and evaluate generated code even when tasks are matched to their skill level (Nguyen et al., 2024). These findings do not settle the longitudinal question developed here, but they make it plausible that artifact performance and human capability should be measured separately.

The paper therefore uses **developmental co-building**, **systems-compositional literacy**, and **the vanishing middle** as provisional analytic terms. Their value depends on whether they isolate a useful configuration of practices and outcomes beyond what is already captured by cognitive apprenticeship, end-user development, mixed-initiative systems, and existing work on AI-assisted programming.

2. The Missing Category

Contemporary discussion frequently represents AI-assisted software construction using two highly legible poles:

Human builds software.

AI builds software.

Between them exists a less clearly articulated practice.

The human does not personally implement most of the system. Neither does the human merely provide an initial specification and evaluate an autonomously produced artifact.

Instead, human and AI repeatedly exchange partial representations of the emerging system.

The human contributes intent, constraints, observations, analogies, corrections, and increasingly sophisticated hypotheses about system structure. The AI contributes implementation knowledge, technical translation, established patterns, feasibility constraints, code generation, and debugging capability.

Crucially, information travels **in both directions**. This reciprocal structure is consistent with mixed-initiative approaches that treat human and automated initiative as dynamically coupled rather than mutually exclusive (Horvitz, 1999).

The AI converts human abstractions into technically actionable structures.

The resulting implementation exposes the human to previously unknown concepts and constraints.

Those concepts become available during subsequent reasoning.

Future human requests therefore differ from earlier requests.

The output of co-building is consequently not only:

artifact_n

but potentially:

artifact_n + increased human systems literacy

which affects:

artifact_(n+1)

That second output is easy to overlook when development systems are evaluated primarily according to artifact quality, task completion, speed, cost, benchmark performance, or degree of autonomy.

3. Building Literacy Without Coding Fluency

Traditional discussions can implicitly bundle several capabilities together:

programming ability = software-building ability

Generative AI partially separates them.

A person may remain unable to independently implement a substantial system while nevertheless becoming increasingly capable of reasoning about:

- component roles and boundaries;
- information and control flow;
- dependencies and runtime requirements;
- state and persistence;
- interfaces between subsystems;
- architectural trade-offs;
- common failure modes;
- reusable mechanisms;
- compatibility constraints; and
- relationships between mechanisms previously encountered in different systems.

This paper uses **systems-compositional literacy** as a provisional label for this capability bundle. The term is intentionally narrower than general end-user development and broader than implementation fluency: it refers to the ability to reason about components, interfaces, constraints, and recombination while relying on AI for much of the implementation. Related work in end-user development and meta-design provides an important conceptual foundation for this possibility (Lieberman, Paternò, & Wulf, 2006; Fischer & Giaccardi, 2006).

It is not programming fluency.

But neither is it merely “having ideas.”

Someone possessing it can increasingly reason in forms such as:

Take mechanism A from one system, mechanism B from another, and mechanism C from a third. Their properties appear compatible under these constraints. None completely supplies the interface required between A and C, so mechanism D must be designed. Here are the properties D needs to preserve. Let us determine whether the resulting composition is coherent.

The AI can then supply implementation knowledge unavailable to the human, challenge the proposed composition, identify hidden constraints, and ultimately instantiate the architecture.

This constitutes a **handshake** between complementary forms of capability.

The human is not merely present to approve output, catch errors, provide oversight, or satisfy a safety requirement. The human may become a more capable participant in construction *through construction*. That possibility is consistent with traditions that treat learning as situated participation and scaffolded practice rather than as the passive receipt of completed answers (Brown, Collins, & Duguid, 1989; Collins, Brown, & Holum, 1991).

4. The Recombination Threshold

Early co-building may primarily involve translating human goals into implementations:

idea -> AI-assisted implementation

With accumulated exposure, however, the human can begin reasoning using mechanisms encountered during previous builds:

known mechanism A + known mechanism B + novel relationship -> proposed system C

Eventually:

A + B + C + newly conceived connector D -> architecture not previously available to the human

At that point AI is not merely lowering the implementation barrier.

It has helped the human acquire enough conceptual machinery to generate richer technical abstractions for the AI to act upon.

This creates a possible positive feedback loop:

AI capability -> human participation -> human capability acquisition -> richer human specification and recombination -> better utilisation of AI capability -> further exposure and learning

Autonomous agents may continue becoming dramatically more capable without necessarily reproducing this developmental pathway. The relevant question is not whether an autonomous system could independently generate the same architecture. The relevant observation is that a human who participated in previous construction may now be able to conceive and direct combinations that were previously outside that human's accessible design space.

The object of interest is therefore not only improvement in the machine or artifact.

It is development of the **human side of the joint system**.

5. Artifact Capability and Human Capability Can Diverge

Consider two users beginning with similar technical knowledge.

One spends an extended period using increasingly autonomous agents. The other spends the same period engaged in sustained co-building.

Both may produce increasingly sophisticated artifacts because AI capability is improving.

Those trajectories do not necessarily imply equivalent changes in human capability.

Conceptually:

Autonomous pathway

Human capability: relatively flat or task-dependent

Artifact capability: rapidly increasing

Developmental co-building pathway

Human capability: potentially increasing through participation

Artifact capability: rapidly increasing

This is a hypothesis, not an established empirical result.

Existing studies establish neither a simple deskilling story nor a simple augmentation story. AI code generators can improve short-term efficiency and reduce workload (Gardella, Pettit, & Riggs, 2024), while reviews and novice studies identify concerns around over-reliance, accuracy, specification, and evaluation (Cambaz & Zhang, 2024; Nguyen et al., 2024). Longitudinal research would be required to determine under what conditions artifact capability and human capability diverge, the magnitude of any divergence, which forms of knowledge transfer occur, and whether particular interface designs strengthen or weaken them.

The distinction nevertheless exposes a measurement problem. Evaluating AI development environments only through artifact quality, completion time, cost, or benchmark performance may miss changes occurring in the human collaborator.

6. Technological Compression

Normally, a new practice has time to become culturally understood.

Developmental human-AI co-building may not.

The progression from weak coding assistance, to capable interactive coding, to increasingly autonomous agentic development has occurred rapidly. The programming-education literature has likewise expanded quickly around LLM-based code generation, with recent reviews already treating over-reliance and learning effects as open design problems (Cambaz & Zhang, 2024). As a result, **developmental co-building may be technologically transitional without being developmentally obsolete.**

Its original *necessity* can disappear while its *value* remains.

If autonomous systems remove the necessity for participation faster than researchers, users, and product designers identify the secondary benefits created by participation, the practice can disappear from view.

The claim is not:

AI eliminated a superior development method.

Rather:

AI capability may remove the instrumental pressure that caused users to discover a practice whose non-instrumental and developmental benefits have not yet been adequately characterised.

This creates the possibility that technological advancement can outrun recognition of the human capabilities generated during an intermediate technological regime.

7. Collapse of the Semantic Middle

Two endpoint categories are increasingly easy to recognise:

manual development <-> autonomous development

Intermediate arrangements may then be described primarily according to their distance from either endpoint:

- AI-assisted coding;
- human supervision;
- partial autonomy;
- human-in-the-loop development.

These labels may fail to capture **developmental co-building as a category with its own objective.**

Developmental co-building is not simply incomplete autonomy.

Human participation may itself be productive.

Collapsing human-AI construction onto a single autonomy axis therefore loses another relevant dimension:

What does participation produce in the participant?

A meaningful middle can consequently become semantically compressed even while the underlying practice remains technically possible.

If newcomers encounter only the categories “learn to code” and “tell agents what to build,” they may never recognise developmental co-building as an available third relationship.

The concern is therefore not merely loss of a workflow.

It is loss of **legible choice**.

8. Preserving Legibility, Not Prescribing Participation

The argument does not imply that autonomous agents are undesirable, that users should inspect every generated line of code, that conventional programming should be preserved for its own sake, that delegation necessarily causes deskilling, or that developmental co-building is optimal for every person or task.

For many users and tasks, autonomous construction may be precisely the appropriate choice.

Instead:

Users should remain able to recognise developmental co-building as a distinct available mode, understand its potential benefits, and deliberately choose it when those benefits align with their objectives.

Preservation does not require preference.

The relevant objective is not to force users into the middle.

It is to prevent the middle from becoming culturally invisible before users can make an informed choice about whether it is valuable to them.

A user interested only in obtaining an artifact may rationally prefer autonomous delegation.

A user interested in developing systems intuition may rationally prefer sustained co-building.

A third user may move dynamically between modes, delegating familiar work while participating closely in novel architectural decisions.

These are different objectives, not stages on a universal hierarchy of development practice.

9. Design Implications

AI development environments can preserve the middle without limiting autonomous capability. This is compatible with long-standing mixed-initiative design principles and with later human-AI interaction guidance that emphasises visibility, control, feedback, and support for correction rather than treating autonomy as the only design objective (Horvitz, 1999; Amershi et al., 2019).

They may support optional:

- inspection of intermediate artifacts and decisions;
- intervention during construction;
- architectural explanations;
- decision visibility;
- conversational reasoning;
- component exploration;
- provenance and receipts;
- checkpoints;
- comparison of alternatives; and
- progressive or selective delegation.

These affordances should not be understood solely as safety brakes or mechanisms for human oversight.

They can also function as **learning surfaces**.

Autonomy itself can become graduated.

A developing user might initially participate heavily, later recognise which operations are routine, and increasingly delegate those operations while remaining closely involved in novel architectural work.

Successful co-building may therefore produce more sophisticated users of autonomous agents.

Such users may become better able to decide what to delegate, what to inspect, when an architecture appears inconsistent with its intended constraints, and how to provide richer specifications.

The capability that makes human participation unnecessary for producing an artifact should not automatically be assumed to make participation valueless.

10. Research Agenda

The claims developed here generate empirical questions rather than settling them.

Future work could investigate:

1. Do sustained AI co-builders acquire systems literacy without comparable increases in implementation fluency?
2. Does developmental co-building improve users' ability to recognise architectural errors?
3. Does it increase cross-domain mechanism transfer and novel recombination?
4. How does autonomous delegation compare with co-building in immediate artifact quality versus later problem formulation and architectural reasoning?
5. Which interface behaviours produce learning rather than passive observation?
6. At what point does participation become unnecessary friction rather than productive exposure?
7. Can users deliberately transition from intensive co-building toward selective autonomy while retaining acquired capability?
8. Which users benefit from developmental co-building, and which rationally gain more from full delegation?
9. How quickly is the cultural vocabulary surrounding AI construction shifting toward autonomous delegation?
10. Does participation in earlier human-AI construction change the kinds of systems a user is subsequently capable of conceiving?

The final question may be particularly important.

If repeated co-building changes not merely how effectively a person can request software, but **what that person becomes capable of imagining as technically constructible**, then artifact-centric evaluations capture only part of the value created by human-AI development systems.

10.1 Limits of Current Evidence

The paper is intentionally hypothesis-generating. It does not establish that sustained co-building reliably produces systems-compositional literacy, that autonomous delegation reliably prevents such development, or that one workflow is superior across users and tasks. Existing research is still dominated by short-term studies, educational settings, specific tools, and rapidly changing model capabilities (Cambaz & Zhang, 2024; Gardella, Pettit, & Riggs, 2024; Nguyen et al., 2024).

The key empirical requirement is therefore longitudinal comparison. Studies would need to separate immediate artifact performance from later unaided reasoning, transfer across projects, architectural error detection, mechanism recombination, and the ability to decide when automation should or should not be trusted. The argument of this paper is narrower: these human-side outcomes are plausible, theoretically grounded, and easy to miss if evaluation focuses only on what the artifact can do.

11. Conclusion

As AI development systems become capable of autonomously producing increasingly sophisticated artifacts, the developmental value of participating in their construction risks becoming culturally invisible.

Human-AI co-building should therefore be recognised as a distinct mode of practice—not because it should be universally preferred, but because sustained participation may develop forms of systems and compositional literacy unavailable through either conventional implementation barriers or artifact-focused autonomous delegation.

The central concern is one of option-space.

Technological advancement may remove the practical necessity for close human participation before the developmental consequences of that participation have been adequately identified. If cultural narratives subsequently compress software construction into a binary choice between manual programming and autonomous AI production, users may lose awareness of a meaningful intermediate practice without ever consciously rejecting it.

Preserving the legibility of this middle maintains informed choice and creates space to study its properties before rapid technological change renders it historically obscure.

The capability that makes human participation unnecessary for producing the artifact may arrive faster than our understanding of what humans gain by participating.

The middle need not be mandatory.

It should remain visible.

References

- Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Paper 3, pp. 1-13). Association for Computing Machinery. <https://doi.org/10.1145/3290605.3300233>
- Brown, J. S., Collins, A., & Duguid, P. (1989). Situated cognition and the culture of learning. *Educational Researcher*, 18(1), 32-42. <https://doi.org/10.3102/0013189X018001032>
- Cambaz, D., & Zhang, X. (2024). Use of AI-driven code generation models in teaching and learning programming: A systematic literature review. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education*, Vol. 1 (pp. 172-178). Association for Computing Machinery. <https://doi.org/10.1145/3626252.3630958>
- Collins, A., Brown, J. S., & Holum, A. (1991). Cognitive apprenticeship: Making thinking visible. *American Educator*, 15(3), 6-11, 38-46.

- Fischer, G., & Giaccardi, E. (2006). Meta-design: A framework for the future of end-user development. In H. Lieberman, F. Paternò, & V. Wulf (Eds.), *End User Development* (pp. 427-457). Springer.
https://doi.org/10.1007/1-4020-5386-X_19
- Gardella, N., Pettit, R., & Riggs, S. L. (2024). Performance, workload, emotion, and self-efficacy of novice programmers using AI code generation. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1* (pp. 290-296). Association for Computing Machinery.
<https://doi.org/10.1145/3649217.3653615>
- Horvitz, E. (1999). Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 159-166). Association for Computing Machinery.
<https://doi.org/10.1145/302979.303030>
- Lieberman, H., Paternò, F., & Wulf, V. (Eds.). (2006). *End User Development*. Springer.
<https://doi.org/10.1007/1-4020-5386-X>
- Nguyen, S., Babe, H. M., Zi, Y., Guha, A., Anderson, C. J., & Feldman, M. Q. (2024). How beginning programmers and code LLMs (mis)read each other. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Article 651, pp. 1-26). Association for Computing Machinery.
<https://doi.org/10.1145/3613904.3642706>
- Papert, S. (1980). *Mindstorms: Children, computers, and powerful ideas*. Basic Books.
- Risko, E. F., & Gilbert, S. J. (2016). Cognitive offloading. *Trends in Cognitive Sciences*, 20(9), 676-688.
<https://doi.org/10.1016/j.tics.2016.07.002>